Kingdoms of Amalur: Re-Reckoning modding documentation

This document lists some of the tools and information disclosed and provided for modding the game. It will be updated via regular game updates.

Tools

pakfileunpacker.exe

This tool can be used to gather information from the proprietary game archive files or unpack contents of them.

Usage : pakfileunpacker pakfilename command

Valid commands are :

list	lists all contents of the given pakfile to stdout
unpack targetfolder [file] [file] ...	unpacks to targetfolder. All files if no filename is given or the specified files.

pakfilebuilder.exe

This tool is used to create the proprietary game archive files. It is not possible to edit an existing archive file, they must always be created from scratch.

Usage : pakfilebuilder [-c] inputlistfile outputfilename

-c enables compression (please always use compression as at the time of writing, uncompressed pakfiles are not supported)

The input list file should have full pathnames of the files to be put into the archive on separate lines.

Data files

Texture files :

Texture files are named '.dds' in the archives for historical reasons.
They are comprised of a header with information about the texture plus the actual pixel data. The pixel data is stored as it would be in a dds file (see information on the internet).

Many textures store multiple levels in separate file. The game will stream in the smallest level usable for the display distance of rendered objects. The textures will have their level stored as a numeric extension of the filename.

```
struct TextureHeader
{
	uint32			assetId;
	uint16			width;				// width and height
of the *largest* variant of the texture set
	uint16			height;
	uint8			variationLevel;			// Note : _1.dds
texture stores a 0, then _2 _3 get the values increased by 1, _0.dds is the last
variationlevel.
	uint8			padding[7];

	uint32			magic;				// 4B45454E

	// from here on this is the header of the actual texture at hand.

	uint16				width;
	uint16				height;

	uint16				depth;		// set to 1
	uint16				flags;		// set to 1 for a texture

	uint8			type;				// TextureType_xxx	2d
= 1, cube = 3
	uint8			format;				// PixelFormat_xxx
	BC1= 1, BC3 = 5
	uint8			lutFormat;			// unused
	uint8			multiSampleType;  // MultiSampleType_xxx  none = 0

	uint8			addressModeU;			// unused in this game
	uint8			addressModeV;			// unused in this game
	uint8			addressModeW;			// unused in this game
	uint8			levelCount;		// 1 for no mipmap

	uint8			cpuAccessMode;			// set to 0
	uint8			gpuAccessMode;	// set to 1
};
```